

Лекция 10.

Балансировка нагрузки и доступность web-сервисов с HAProxy

Цель лекции

Понять, зачем web-сервису нужен reverse proxy и load balancer

Разобрать базовую структуру HAProxy: global, defaults, frontend, backend, server

Настроить HAProxy перед двумя Nginx backend

Добавить HTTP Health Check через /health

Доказать работу сервиса при отказе одного backend и недоступность при отказе всех backend

Учебные вопросы

1. SPOF, вертикальное и горизонтальное масштабирование
2. Reverse Proxy и Load Balancer
3. HAProxy frontend, backend и server
4. Round Robin и HTTP Health Checks
5. Таймауты и базовая статистика
6. Отказ backend, обзор TLS Termination и SPOF самого HAProxy
7. Базовый мониторинг через Zabbix

Учебная топология одна для всей лекции

client01: 192.168.10.10

lb01: HAProxy, 192.168.30.10

web01: Nginx, 192.168.30.11, ответ Response from web01

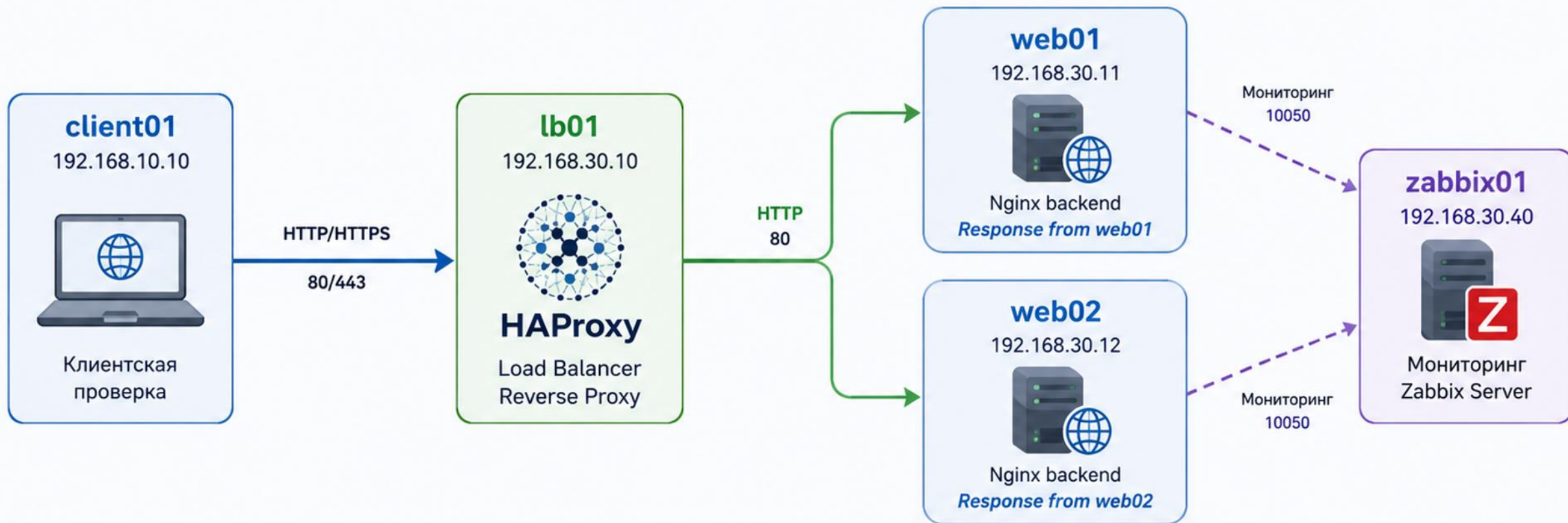
web02: Nginx, 192.168.30.12, ответ Response from web02

zabbix01: 192.168.30.40

Клиент обращается только к lb01, а не напрямую к web01 и web02

Топология: client01, lb01, web01, web02 и zabbix01

Учебный стенд: клиент отправляет запросы на HAProxy (lb01), который балансирует трафик между backend-серверами web01 и web02.



Роли	Сети
client01 – клиентская проверка	192.168.10.0/24 – клиентская сеть
lb01 – HAProxy (балансировка)	192.168.30.0/24 – серверная сеть
web01 – Nginx backend	
web02 – Nginx backend	
zabbix01 – мониторинг	

Поток запросов
Клиент → HAProxy (HTTP/HTTPS 80/443)
HAProxy → backend (HTTP 80)
Мониторинг (Zabbix agent 10050)

Важно
✓ На web01 и web02 должны быть разные страницы: Response from web01 и Response from web02
✓ HAProxy распределяет трафик между backend-серверами (Round Robin по умолчанию)

Единый учебный сценарий HAProxy

Подготовить web01 и web02

Настроить HAProxy frontend и backend

Включить Round Robin

Добавить HTTP Health Check

Проверить распределение curl-запросами

Смоделировать отказ web01, восстановление, HTTP 500 и отказ обоих backend

1. Один web-сервер становится SPOF

SPOF (Single Point of Failure) - единственная точка отказа

Если один web-сервер обслуживает весь сервис, его отказ останавливает доступ пользователей

Перезагрузка, ошибка конфигурации или отказ диска сразу становятся проблемой сервиса

SPOF устраняют не резервной копией, а избыточностью в рабочей схеме

Для web-уровня базовое решение - несколько backend за load balancer

Backup, Monitoring и Load Balancer решают разные задачи

Backup помогает восстановить данные позже

Monitoring помогает узнать, что проблема уже появилась

Load Balancer продолжает обслуживать запросы через исправный backend

Эти механизмы не заменяют друг друга

Для доступности нужны и резервные копии, и мониторинг, и отказоустойчивая схема обслуживания

Vertical Scaling не устраняет SPOF

Vertical Scaling - увеличить CPU, RAM или диск одного сервера

Такой сервер может выдержать больше нагрузки

Но при его отказе сервис всё равно остановится

Horizontal Scaling - добавить несколько серверов одного уровня

HAProxy распределяет запросы между несколькими backend и снижает риск отказа одного узла

д Balancer не делает всё приложение отказоустойчивым автоматически

web01 и web02 могут зависеть от одной базы данных
Если shared Database отказала, оба backend могут
перестать работать

HAProxy защищает только тот уровень, где есть
несколько исправных backend

Состояние приложения, база данных, файлы и сессии
проектируются отдельно

Главный вывод: балансировка web-уровня - важный,
но не единственный элемент HA

2. Reverse Proxy и Load Balancer

Reverse Proxy принимает запрос клиента и передает его внутреннему серверу

Клиент видит адрес проху, а не конкретный backend

Load Balancer выбирает один backend из нескольких

HAProxy может выполнять обе роли одновременно

В лаборатории HAProxy принимает HTTP-запросы на lb01 и распределяет их между web01 и web02

Путь запроса через HAProxy должен быть понятен

Client отправляет запрос на 192.168.30.10:80

HAProxy принимает запрос во frontend

HAProxy выбирает backend-сервер по алгоритму
balance

web01 или web02 формирует HTTP-ответ

Клиент получает ответ через HAProxy и не выбирает
backend вручную

3. HAProxy строится из frontend, backend и server

global - общие параметры процесса HAProxy

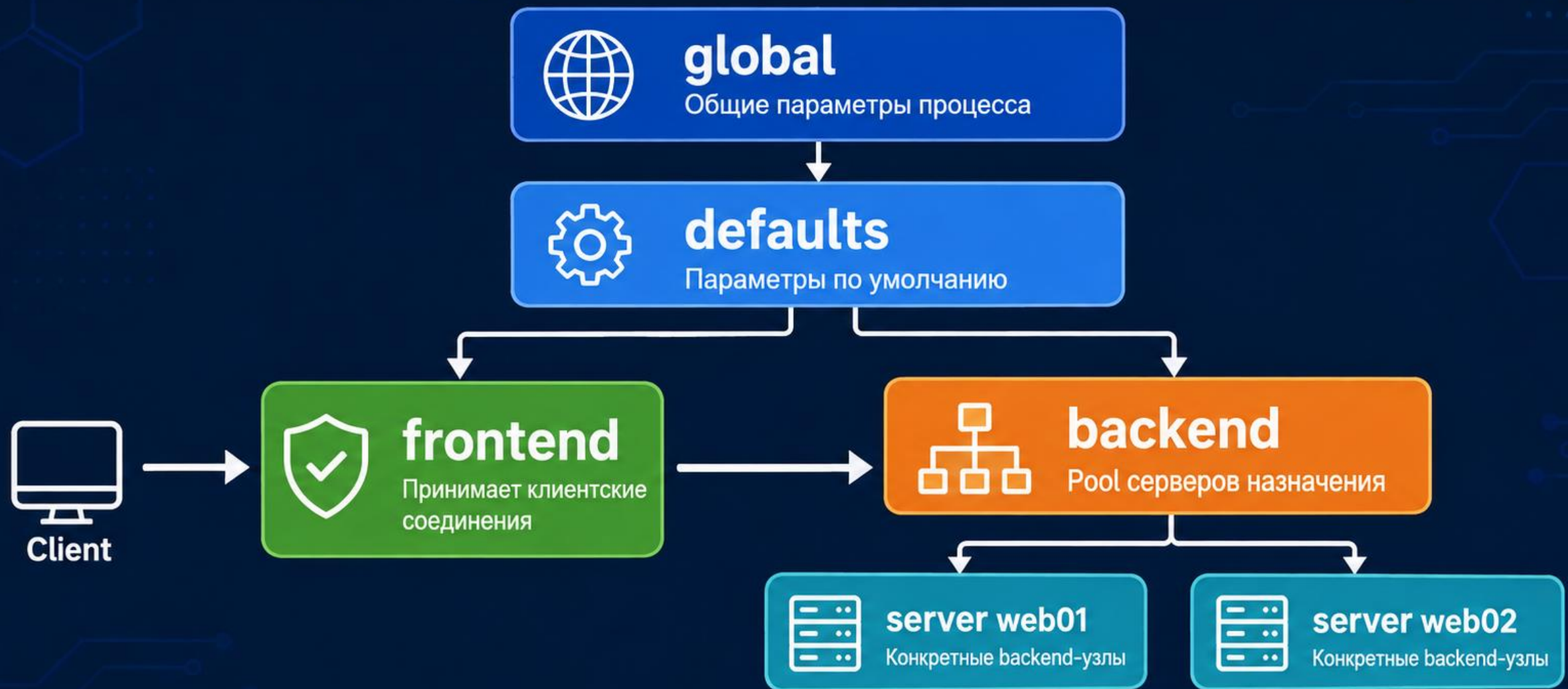
defaults - настройки по умолчанию для HTTP-прокси

frontend - точка входа для client-запросов

backend - pool серверов, куда передаются запросы

server - конкретный backend-узел: имя, IP, порт и
параметры проверки

Структура конфигурации HAProxy



frontend → принимает запрос



backend → выбирает pool



server → конкретный сервер

mode http нужен для учебной HTTP-балансировки

mode tcp работает на уровне TCP-соединений

mode http понимает HTTP-запросы, заголовки и коды ответа

HTTP Health Check проверяет не только открытый порт, но и ответ приложения

В лаборатории используется mode http

L4-балансировка рассматривается обзорно, без настройки в этой лекции

HAProxy устанавливается и настраивается на lb01

```
sudo apt update
```

```
sudo apt install -y haproxy
```

Основной файл: `/etc/haproxy/haproxy.cfg`

Перед применением конфигурации файл обязательно проверяется

После успешной проверки выполняют `reload`, а не слепой `restart`

Backend Nginx должен отдавать разные страницы

На web01: `echo 'Response from web01' | sudo tee /var/www/html/index.html`

На web02: `echo 'Response from web02' | sudo tee /var/www/html/index.html`

Прямой тест web01: `curl http://192.168.30.11/`

Прямой тест web02: `curl http://192.168.30.12/`

Разные ответы позволяют увидеть Round Robin глазами студента

/health должен возвращать HTTP 200, когда backend готов

Health endpoint - короткий URL для проверки готовности backend

Для лаборатории используется GET /health

Нормальный ответ: HTTP 200

Пример Nginx location: location = /health { return 200 'ok'; }

После изменения Nginx проверяют: `sudo nginx -t &&`
`sudo systemctl reload nginx`

Полный минимальный haproxy.cfg: global и defaults

global

log /dev/log local0

defaults

log global

mode http

option httplog

timeout connect 5s

timeout client 30s

timeout server 30s

Полный минимальный haproxy.cfg: frontend и backend

```
frontend fe_http
    bind 192.168.30.10:80
    default_backend be_web
backend be_web
    balance roundrobin
    option httpchk GET /health
    http-check expect status 200
    server web01 192.168.30.11:80 check inter 2s fall 3 rise 2
    server web02 192.168.30.12:80 check inter 2s fall 3 rise 2
```

Конфигурация проверяется до reload

`sudo haproxy -c -f /etc/haproxy/haproxy.cfg`

Ожидаемый результат: Configuration file is valid

Только после этого: `sudo systemctl reload haproxy`

Проверка процесса: `systemctl status haproxy`

Если `haproxy -c` показывает ошибку, reload выполнять нельзя

bind задает адрес и порт frontend

bind 192.168.30.10:80 слушает только нужный Service IP

bind *:80 слушает все подходящие адреса сервера

Для учебной схемы лучше явно указать

192.168.30.10:80

Проверка прослушивания: ss -ltnp | grep :80

Если frontend не слушает порт, клиент не попадет к backend

4. Round Robin распределяет запросы по очереди

Round Robin - базовый алгоритм балансировки

Request 1 → web01

Request 2 → web02

Request 3 → web01

Request 4 → web02

Алгоритм удобен для первой лаборатории, потому что результат легко увидеть

Round Robin проверяется серией curl-запросов

```
for i in {1..10}; do curl -s -H 'Connection: close'  
http://192.168.30.10/; done
```

Connection: close уменьшает влияние повторного использования соединения

Ожидаемые ответы: Response from web01 и Response from web02

Если ответы не чередуются, сначала проверяют backend status и haproxy.cfg

Проверка должна выполняться через lb01, а не напрямую к backend

Least Connections и weight оставляем как обзор

leastconn выбирает backend с меньшим числом активных соединений

weight меняет долю запросов для конкретного server

Эти параметры полезны в реальных системах

В первой лаборатории они не обязательны

Основной алгоритм занятия - balance roundrobin

Health Check не дает отправлять клиентов на сломанный backend

Без проверки HAProxy может отправить запрос на неработающий backend

check включает проверку состояния server

Исправный backend остается в pool

Сломанный backend временно исключается из pool

Центральный результат лекции - доказать это экспериментом

TCP Check и HTTP Check доказывают разные вещи

TCP Check: порт 80 открыт, значит backend может считаться UP

Но приложение может ответить HTTP 500

HTTP Check: HAProxy запрашивает /health
option httpchk GET /health

http-check expect status 200

Для web-сервиса HTTP Check полезнее простого TCP Check

inter, fall и rise задают поведение проверки

inter 2s - выполнять check каждые 2 секунды

fall 3 - перевести backend в DOWN после 3 неудачных checks

rise 2 - вернуть backend в UP после 2 успешных checks

Эти параметры защищают от резких переключений при кратких сбоях

Студент должен уметь объяснить, почему backend не исчезает мгновенно

Цикл состояния backend в HAProxy



1



UP →

сервер получает трафик

2



fall →

сколько ошибок нужно для DOWN

3



rise →

сколько успешных проверок нужно для возврата в UP

Sessions и Sticky Sessions важны, но не являются базовой настройкой

Если session хранится только на web01, следующий запрос на web02 может потерять состояние
Backend должны быть одинаково готовы обслужить запрос

Sticky Session закрепляет client за backend

Sticky Sessions - компромисс, а не полное решение проблемы состояния

В этой лекции настройка sticky не является обязательной

5. Таймауты защищают HAProxy от зависших соединений

`timeout connect 5s` - время ожидания соединения с backend

`timeout client 30s` - ожидание активности со стороны client

`timeout server 30s` - ожидание ответа backend

Слишком большие таймауты держат ресурсы занятыми

Слишком маленькие таймауты могут обрывать нормальные запросы

Stats показывает состояние pool, но не должен быть публичным

Stats показывает backend UP/DOWN и connections

Для лаборатории stats можно привязать к loopback или management IP

Пример: `bind 127.0.0.1:8404`

Пример: `stats uri /haproxy?stats`

Stats page помогает преподавателю и студенту быстро увидеть результат health checks

Logs HAProxy помогают понять reload и отказы

`journalctl -u haproxy -b`

`journalctl -u haproxy -b --since '-5 min'`

После reload проверяют, появились ли ошибки конфигурации

При отказе backend logs помогают подтвердить переход server в DOWN

Централизованный Syslog изучается отдельно

Таблица: проверка и что она доказывает

`systemctl status haproxy` → процесс HAProxy работает

`haproxy -c` → синтаксис конфигурации корректен

`ss -ltnp` → frontend слушает порт

`curl http://192.168.30.11/` и `curl http://192.168.30.12/` →

backend работают напрямую

`curl http://192.168.30.10/` → сервис работает через HAProxy

`/health`, `Stats` и `Zabbix HTTP` → backend ready, pool

UP/DOWN и пользовательская доступность

6. Проверка отказа начинается с нормального состояния

web01 UP

web02 UP

Серия curl показывает Response from web01 и Response from web02

Stats показывает оба backend в состоянии UP

Только после базовой проверки есть смысл имитировать отказ

Отказ web01 не должен остановить сервис

На web01: `sudo systemctl stop nginx`

После fail 3 HAProxy должен перевести web01 в DOWN
web02 остается UP

Клиент продолжает получать Response from web02

Это главный эксперимент: один backend отказал, но сервис продолжает отвечать

Recovery web01 возвращает backend в pool

На web01: `sudo systemctl start nginx`

После rise 2 HAProxy должен вернуть web01 в UP

Серия `curl` снова показывает ответы от web01 и web02

Stats подтверждает восстановление backend

Студент фиксирует время и наблюдаемое состояние в отчете

HTTP 500 при открытом TCP 80 показывает ценность HTTP Check

TCP 80 может оставаться открытым

Но /health может вернуть HTTP 500

Пример Nginx location: location = /health { return 500
'fail'; }

После reload Nginx HAProxy должен исключить backend
через HTTP Check

Так студент видит разницу между доступностью порта
и готовностью приложения

Отказ обоих backend должен дать HTTP 503

Остановить Nginx на web01 и web02

Проверка: `curl -i http://192.168.30.10/`

Ожидаемо: HTTP 503 Service Unavailable

1 backend DOWN → degradation

2 backend DOWN → service unavailable

Graceful Drain оставляем как дополнительный сценарий

Planned Maintenance - плановое обслуживание backend

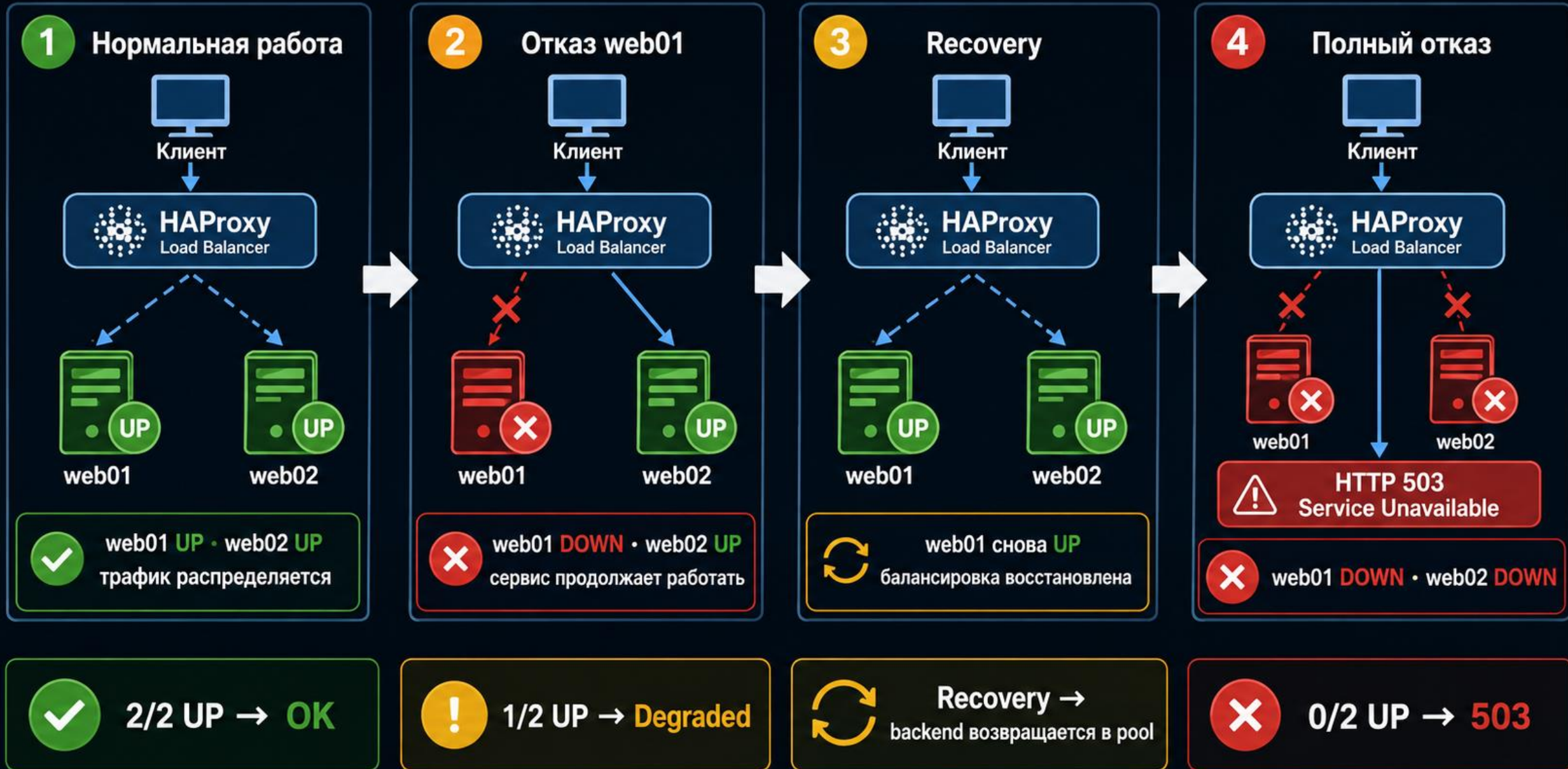
Backend убирают из pool до обновления

Администратор дожидается завершения активных соединений

После обновления backend проверяют и возвращают в pool

Подробная настройка drain не входит в обязательную часть Junior-лабы

Сценарии работы backend в HAProxy



TLS Termination нужно понять обзорно

TLS Termination - HAProxy принимает HTTPS от клиента

Дальше HAProxy может отправлять HTTP к backend

Пример: `bind 192.168.30.10:443 ssl crt
/etc/haproxy/certs/site.pem`

Это удобно для централизованного управления
сертификатом

Полноценная TLS-лаборатория не является
обязательной в этой лекции

X-Forwarded-For и X-Forwarded-Proto объясняются упрощенно

Backend обычно видит адрес HAProxy, а не исходный client IP

option forwardfor добавляет X-Forwarded-For

Backend должен доверять этому заголовку только от своего HAProxy

После TLS Termination полезен X-Forwarded-Proto

Пример: http-request set-header X-Forwarded-Proto

https if { ssl_fc }

PROXY Protocol и продвинутый TLS относятся к дополнительным темам

PROXY Protocol может передавать client IP для L4 proxy

В основной HTTP-лаборатории он не используется

SNI, несколько сертификатов и Backend HTTPS

изучаются позже

openssl s_client и CA verification также относятся к

расширенной диагностике

Junior сначала должен уверенно настроить HTTP-

балансировку и health checks

Один HAProxy тоже может стать SPOF

web01 и web02 могут быть исправны

Но если lb01 отказал, client не попадет к backend

Load Balancer устраняет SPOF web-сервера, но сам требует отказоустойчивости

Это следующий архитектурный уровень после базовой лаборатории

Главный вывод: один балансировщик тоже является SPOF

Keepalived и VIP рассматриваются обзорно

VIP (Virtual IP) - общий адрес пары балансировщиков

Keepalived / VRRP переносит VIP между Ib01 и Ib02

Ib01 и Ib02 должны иметь одинаковую проверенную конфигурацию

VirtualBox и облачные среды могут иметь ограничения multicast, ARP и floating IP

Настройка Keepalived не входит в обязательную часть этой лекции

7. Zabbix Monitoring фиксирует состояние сервиса

Zabbix не заменяет HAProxy Health Check

Zabbix показывает проблему преподавателю и администратору

Минимум для lb01: HAProxy service

Минимум для сервиса: Frontend HTTP

Минимум для pool: количество backend UP

Degradation и Failure нужно различать в мониторинге

2/2 Backend UP → OK

1/2 Backend UP → Degraded

0/2 Backend UP → Service Down

Для Zabbix это разные триггеры и разные приоритеты

Пользователь может еще видеть сервис при Degraded,
но администратор уже должен реагировать

Zabbix проверяет не только процесс, но и пользовательский HTTP

Process check: haproxy запущен на lb01

Port check: 192.168.30.10:80 слушает

HTTP check: http://192.168.30.10/ возвращает ответ

Backend status: web01/web02 UP или DOWN

Stats page не должна быть единственным
доказательством доступности сервиса

Основной лабораторный цикл: настройка

1. Установить Nginx на web01 и web02
2. Создать разные index pages и /health
3. Проверить оба backend напрямую
4. Установить HAProxy на lb01
5. Настроить global, defaults, frontend и backend
6. Настроить Round Robin и HTTP Health Check
7. Выполнить haproxy -c, reload и серию curl через lb01

Основной лабораторный цикл: отказы и приемка

8. Остановить web01 и доказать продолжение сервиса через web02
9. Вернуть web01 и дождаться Recovery
10. Сделать /health = 500 при открытом TCP 80
11. Остановить оба backend и получить HTTP 503
12. Проверить logs, Stats и Zabbix-состояние
13. Сохранить рабочий haproxy.cfg в Git
14. Объяснить, что доказала каждая проверка

Типовые ошибки HAProxy в базовой лаборатории

mode http отсутствует или задан не там

bind указан на неверный IP или порт уже занят

backend IP или port указан неправильно

/health не возвращает HTTP 200

haproxy -s не выполнен перед reload

firewall блокирует доступ от lb01 к backend

web01 и web02 содержат разные версии приложения
без контроля

Итоги лекции

HAProxy принимает клиентские запросы во frontend и передает их в backend pool

Round Robin позволяет увидеть базовое распределение нагрузки

HTTP Health Check исключает backend, если приложение не готово отвечать

Отказ одного backend должен приводить к degradation, а отказ всех backend - к HTTP 503

TLS, Keepalived, PROXY Protocol и сложный monitoring изучаются после уверенной базовой настройки

Контрольные вопросы

1. Что такое SPOF?
2. Чем Reverse Proxy отличается от Load Balancer?
3. Что делают frontend, backend и server в HAProxy?
4. Как работает Round Robin?
5. Для чего нужен Health Check?
6. Чем TCP Check отличается от HTTP Check?
7. Что означают inter, fall и rise?
8. Что происходит при отказе одного backend?
9. Что происходит при отказе всех backend?
10. Зачем выполнять haproxy -s перед reload?
11. Почему один HAProxy тоже остается SPOF?

Схема: frontend, backend и server в HAProxy

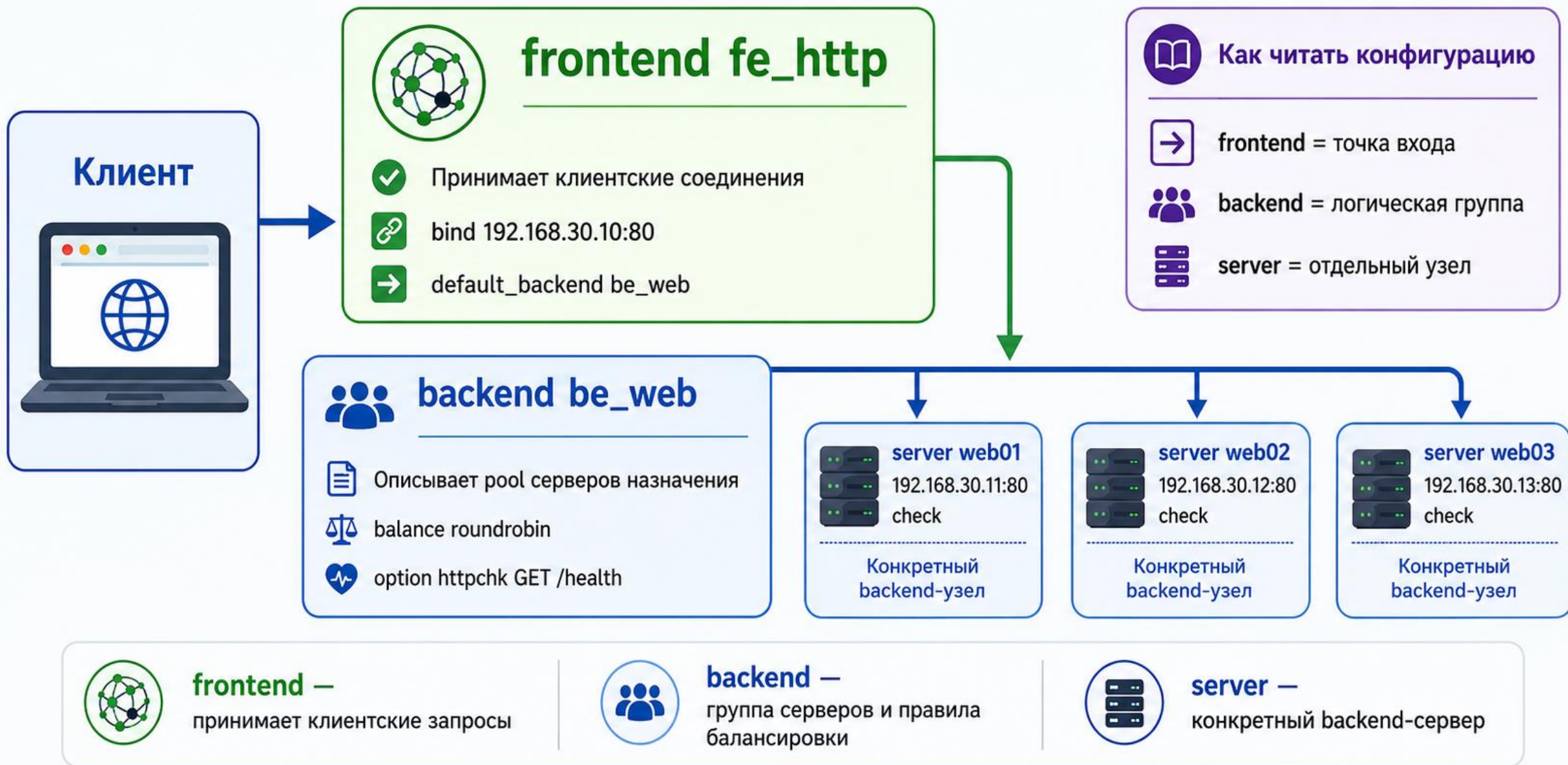


Схема: HAProxy и обзор Keepalived/VIP

